

DOCUMENTATION SERVICES DOCUMENTATION

Presentado por:

Isabella Garces Acosta

Diego Steven Pinzon Yossa

Juan Camilo Rosero Satisfesteban

Sharick Yelixa Torres Monroy

Laura Sofia Vargas Rodriguez

Profesora:

Liliana Marcela Olarte Mesa



**Universidad Nacional de Colombia
Facultad de ingeniería
Departamento de Ingeniería de sistemas e industrial
2025**



CONTENT

- 1. Amadeus Service**
- 2. Hotels Service**
- 3. Itinerary Generator Service**
- 4. Flights Service**
- 5. Places Service**
- 6. User Service**
- 7. Restaurants Service**



AMADEUS SERVICE

This comprehensive service (`amadeus.service.ts`) acts as the primary gateway and abstraction layer for interacting with the Amadeus Travel APIs, providing structured access to flight, hotel, and airport data through a unified interface. The service implements sophisticated token management with automatic expiration handling and refresh logic, ensuring efficient API usage while maintaining secure authentication. Key features include typed API responses for hotels (`AmadeusHotelListResponse`, `AmadeusHotelSearchResponse`), airports (`AmadeusAirportResponse`), and flights (`AmadeusFlightOffersResponse`) with full TypeScript support. The service exposes clean methods like `searchHotelsByGeocode()`, `searchHotelOffers()`, `searchNearestAirport()`, and `searchFlightOffers()` that abstract away the complexity of Amadeus' API structure. It includes intelligent error handling, proper parameter validation, and comprehensive logging for debugging. This service forms the foundation for all Amadeus-powered travel data in the application, providing reliable access to real-time availability, pricing, and location information while handling the intricacies of travel industry APIs.

HOTEL SERVICE

This hotels service (`hotels.service.ts`) serves as the business logic layer that transforms raw Amadeus API data into the application's standardized hotel format. It orchestrates complex multi-step hotel searches: first retrieving hotels by geographic coordinates, then fetching real-time availability and pricing offers, and finally applying business rules for filtering and ranking. The service includes intelligent batching for large hotel lists (respecting Amadeus' 30-hotel limit per request), rate limiting with configurable delays, and sophisticated data transformation that enriches hotel information with pricing details, room specifications, and cancellation policies. It implements filtering by price ranges, hotel chains, and custom business logic while maintaining the application's internal data models. The transformation logic carefully handles missing data fields and provides fallback values, ensuring robustness even with incomplete API responses. This service demonstrates professional API consumption patterns with error resilience, performance optimization, and clean separation between external API data and internal application models.

ITINERARY GENERATOR SERVICE

This itinerary generator service (`itinerary-generator.service.ts`) is the central intelligence engine of the entire travel planning application, orchestrating the complex process of creating complete, day-by-day travel itineraries from user preferences. The service follows a sophisticated multi-stage workflow:

1. Multi-Source Data Collection: Simultaneously queries four specialized APIs (flights, hotels, restaurants, tourist sites) with intelligent parameter tuning based on trip duration and type.
2. Intelligent Selection Algorithms: Implements business logic for selecting optimal flights (prioritizing fewer stops → lower price → shorter duration) and hotels (availability → best value).
3. Temporal Itinerary Construction: Generates realistic daily schedules considering:
 - Travel time calculations and transfer buffers (30 minutes between activities)
 - Meal timing logic (breakfast 8:00, lunch 12:30, dinner 19:00)
 - Activity duration estimates based on category (museums=3h, parks=2h, etc.)
 - Special day logic for arrival/departure days
4. Dynamic Resource Allocation: Uses circular indexing to distribute restaurants and tourist sites across days without repetition, with intelligent fallbacks when specific categories are unavailable.
5. Financial Calculations & Currency Handling: Converts all prices to COP, calculates per-person costs, and maintains budget constraints while estimating realistic expenses for meals and entrance fees.

The service demonstrates exceptional architectural design with:

- Clean separation of concerns: Distinct methods for each itinerary component type
- Realistic scheduling: Time buffers, realistic activity durations, and natural flow
- Error resilience: Fallback mechanisms at every stage
- Multi-variant generation: Can produce 3+ different itinerary versions from the same data
- Comprehensive logging: Detailed console output for debugging complex generation logic

This service transforms raw travel data into coherent, executable travel plans that balance user preferences, logistical constraints, and budget considerations—representing the core value proposition of the entire application.

FLIGHTS SERVICE

The `flights.service.ts` file implements the `FlightsService` class, which orchestrates real-time flight search operations using the Amadeus API. The core functionality resides in the `searchFlights` method, which executes a six-step process: it first discovers the nearest airports to given geographic coordinates, validates their existence within a specified radius, then queries Amadeus for flight offers based on travel parameters (dates, passengers, cabin class, stop preferences). The service transforms complex API responses into standardized application models through private methods `transformAirportResponse` and `transformFlightOffersResponse`, which map airline codes



to human-readable names using dictionaries and configuration lookups. It applies additional client-side filtering for maximum stopovers and price ranges, sorts results by total cost, and enforces result limits. The class handles both outbound and inbound itineraries, parses pricing structures including base fares, fees and taxes, and maps Amadeus cabin class codes to application-specific enums.

PLACES SERVICE

The `places.service.ts` file provides the `PlacesService` class for discovering tourist attractions via the Google Places API (New). The main method `searchTouristSites` accepts geographic coordinates and iterates through predefined categories (museums, parks, monuments, historical sites), performing parallel searches for each type while deduplicating results using a `Set` to track unique place IDs. The service transforms Google Places responses into standardized `TouristSiteResponse` objects through the `transformGooglePlace` method, which extracts essential data including coordinates, ratings, photo references, and opening hours. It supports configurable search radius, minimum rating thresholds, and category-specific type mappings defined in `PLACES_CONFIG`. The class includes a `getPhotoUrl` method for generating authenticated image URLs and handles edge cases where Google returns inconsistent data structures.

USER SERVICE

The `user.service.ts` file contains utility functions for user profile management and image handling. The `updateUserProfile` function sends a `PUT` request to an internal API endpoint with Firebase `UID` and updateable fields (name and profile image), throwing descriptive errors for failed requests. For image processing, `imageToBase64` asynchronously converts image files to Data URLs using `FileReader`, while `isValidImageFile` validates file type against a whitelist (JPEG, PNG, WebP, GIF) and enforces a 5MB size limit, throwing custom errors for invalid inputs. These functions collectively enable client-side profile updates with robust file validation and conversion.

RESTAURANT SERVICE

The `restaurants.service.ts` file implements the `RestaurantsService` class, which searches dining establishments using the Google Places API with restaurant-specific logic. Similar to the Places service, it categorizes searches using `RESTAURANT_CATEGORY_TYPES` from configuration, but extends functionality by extracting cuisine types from Google's type taxonomy through the `extractCuisineTypes` method. The `searchRestaurants` method supports filtering by price levels and minimum ratings, deduplicates results across categories, and sorts by rating. The `transformGooglePlace` method enriches response objects with cuisine metadata and business information while maintaining the same transformation patterns for photos, opening hours, and geographic data as its tourist-site counterpart.